On the Algorithmic and Mathematical Foundations of Combinatorics: A Formal Analysis of the Binomial Theorem, Multinomial Theorem, and Pascal's Triangle

Abstract

This paper presents a formal analysis of the fundamental principles of combinatorics, focusing on Pascal's Triangle, the Binomial Theorem, and the Multinomial Theorem. We explore the recursive mathematical properties of Pascal's Triangle, formulate the algebraic expansion of binomial and multinomial expressions, and discuss their critical applications in algorithmic design, probabilistic analysis, and dynamic programming optimization. Complete Java reference implementations are provided for efficient algorithmic generation.

Index Terms

binary trees, algorithms, data structures, computational complexity, tree traversal, recursive algorithms, formal analysis

1 Introduction

Combinatorics serves as the bedrock of theoretical computer science, discrete mathematics, and statistical analysis. In competitive programming (CP) and technical entrance examinations like the Graduate Aptitude Test in Engineering (GATE), combinatorial math forms a significant portion of the syllabus. The study of counting principles, selection models, and polynomial expansions provides the critical machinery required for analyzing the time complexity of algorithms, optimizing database query operations, structuring error-correcting codes, and developing machine learning frameworks. Within this mathematical domain, the Binomial Theorem, Pascal's Triangle, and the Multinomial Theorem represent a highly interconnected triad of concepts.

For students and competitive programmers aiming to master these topics, this paper serves as an exhaustive, graduate-level reference manual. It bridges the gap between pure algebra and concrete algorithmic implementations, detailing the mathematical foundations and the optimized code patterns used to solve a vast range of problems.

The structure of this paper is organized as follows:

1) Fundamental Counting and Selection Principles (§2): A formal treatment of the sum and product rules, permutations, and combinations, concluding with a rigorous algebraic proof of Pascal's Identity.
2) Pascal's Triangle: Algebraic Invariants and Algorithmic Complexity (§3): Analysis of Pascal's Triangle, algebraic row sum invariants, and three distinct algorithmic paradigms (naive recursion, 2D dynamic programming, and space-optimized dynamic programming) accompanied by their asymptotic complexity bounds and complete Java reference implementations.
3) The Binomial Theorem and Mathematical Proofs (§4): Detailed mathematical statements, a formal proof using mathematical induction, a combinatorial proof, analysis of middle terms, constant terms, rational terms, Vandermonde's Identity, numerically greatest terms, and various algebraic corollaries (e.g., alternating sums, derivative, and integration-based identities).
4) The Multinomial Theorem and Multiset Permutations (§5): Generalization of the binomial expansion to m variables, derivation of multinomial coefficients via multiset arrangements, the sum of multinomial coefficients, and a formal proof of the number of terms in a multinomial expansion using the "Stars and Bars" combinatorial method.
5) Modular Combinatorics in Competitive Programming (§6): Precomputation of factorials and inverse factorials under modulo $10^9 + 7$ for $O(1)$ combinations, Fermat's Little Theorem, and modular division.
6) Advanced Counting Sequences: Catalan Numbers, Derangements, and Burnside's Lemma (§7): Formulations, proofs, and applications of Catalan Numbers, Derangements, and Burnside's Lemma for symmetrical counting.
7) Lucas' Theorem for Large Combinations (§8): Solving combinations where $N, K \leq 10^{18}$ and modulo P is a small prime.
8) Stirling Numbers of the First and Second Kind (§9): Set partitioning and cycle arrangements, their recurrence relations, and dynamic programming algorithms.
9) Integer Partitions (§10): Decomposing integers, recurrence relations, and dynamic programming algorithms.

10) The Inclusion-Exclusion Principle (PIE) (§11): Formal union cardinality formulation and alternating sign proofs.
11) The Pigeonhole Principle (PHP) (§12): Simple and generalized formulations, and application to subarray sum divisibility.
12) The Twelvefold Way Classification (§13): Systematic mapping of n balls into k boxes.
13) Exhaustive Problem Walkthroughs (§14): Step-by-step mathematical and algorithmic solutions to 12 classic GATE and CP problems.

2 Fundamental Counting and Selection Principles

We begin by establishing the formal definitions and axioms of counting. All combinatorial counting models can be traceably reduced to two fundamental rules of set theory.

2.1 The Sum and Product Rules

Definition 1 (The Sum Rule). Let $S_1, S_2, \ldots, S_k$ be pairwise disjoint finite sets. The cardinality of their union is the sum of their individual cardinalities:

$$\left| \bigcup_{i=1}^{k} S_i \right| = \sum_{i=1}^{k} |S_i| \tag{1}$$

In practical terms, if an event can occur in m ways and a second event can occur in n ways (where the two events are mutually exclusive), there are $m + n$ ways to choose one of the events.

Definition 2 (The Product Rule). Let $S_1, S_2, \ldots, S_k$ be finite sets. The cardinality of their Cartesian product is the product of their individual cardinalities:

$$|S_1 \times S_2 \times \cdots \times S_k| = \prod_{i=1}^{k} |S_i| \tag{2}$$

In practical terms, if a process can be broken down into k successive, independent stages, where stage i can be completed in n_i ways, the entire process can occur in $n_1 \times n_2 \times \cdots \times n_k$ ways.

2.2 Permutations and Combinations

From the product rule, we can derive the formula for counting arrangements of discrete objects.

Definition 3 (Permutation). A permutation is an ordered arrangement of r distinct elements chosen from a set containing n distinct elements. The number of such permutations is denoted by $P(n, r)$ or ${}^nP_r$, and is formulated as:

$$P(n, r) = \prod_{i=0}^{r-1} (n - i) = \frac{n!}{(n - r)!} \quad \text{for } 0 \leq r \leq n \tag{3}$$

Definition 4 (Combination). A combination is an unordered selection of r elements chosen from a set containing n distinct elements. The number of such combinations is denoted by $C(n, r)$, $\binom{n}{r}$, or ${}^nC_r$, and is formulated as:

$$\binom{n}{r} = \frac{P(n, r)}{r!} = \frac{n!}{r!(n - r)!} \quad \text{for } 0 \leq r \leq n \tag{4}$$

2.3 Pascal's Identity

Pascal's Identity provides the fundamental recursive link between adjacent binomial coefficients. We present a complete algebraic proof below.

Theorem 5 (Pascal's Identity). For any positive integers n and k such that $1 \leq k \leq n$:

$$\binom{n}{k} = \binom{n-1}{k-1} + \binom{n-1}{k} \tag{5}$$

Proof. By expanding the right-hand side of the identity using the definition of binomial coefficients, we obtain:

$$\binom{n-1}{k-1} + \binom{n-1}{k} = \frac{(n-1)!}{(k-1)!\,[(n-1)-(k-1)]!} + \frac{(n-1)!}{k!(n-1-k)!} \tag{6}$$

Simplifying the factorial term in the first denominator:

$$= \frac{(n-1)!}{(k-1)!(n-k)!} + \frac{(n-1)!}{k!(n-1-k)!} \tag{7}$$

To add these fractions, we establish a common denominator of $k!(n-k)!$. We multiply the numerator and denominator of the first fraction by k, and the second fraction by $(n-k)$:

$$= \frac{k \cdot (n-1)!}{k \cdot (k-1)!(n-k)!} + \frac{(n-k) \cdot (n-1)!}{k!(n-k) \cdot (n-1-k)!} \tag{8}$$

$$= \frac{k(n-1)!}{k!(n-k)!} + \frac{(n-k)(n-1)!}{k!(n-k)!} \tag{9}$$

Combining the numerators over the common denominator:

$$= \frac{[k + (n-k)](n-1)!}{k!(n-k)!} \tag{10}$$

$$= \frac{n \cdot (n-1)!}{k!(n-k)!} \tag{11}$$

$$= \frac{n!}{k!(n-k)!} = \binom{n}{k} \tag{12}$$

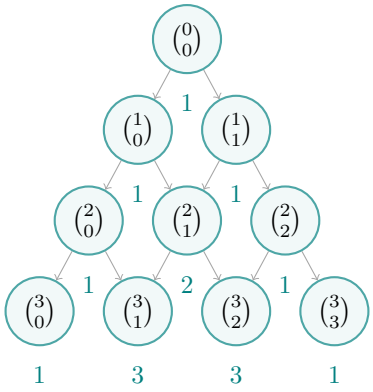
This completes the algebraic proof. □

3 Pascal's Triangle: Algebraic Invariants and Algorithmic Complexity

Pascal's Triangle is a geometric arrangement of the binomial coefficients. Each row n (indexed starting from 0) contains $n+1$ elements corresponding to $\binom{n}{k}$ for $k \in \{0, 1, \ldots, n\}$.

3.1 TikZ Visualization

Below is a graphical representation of the first five rows of Pascal's Triangle (Rows 0 to 4), showing both the binomial coefficient notation and their evaluated values.



3.2 Key Invariants and Properties

Pascal's Triangle exhibits several core mathematical invariants:

1) Symmetry: Within any row n, the coefficients are symmetric about the center:

$$\binom{n}{k} = \binom{n}{n-k} \tag{13}$$

2) Sum of Row Elements: The sum of all elements in row n is 2^n:

$$\sum_{k=0}^{n} \binom{n}{k} = 2^n \tag{14}$$

3) Alternating Row Sum: The alternating sum of elements in row n is 0 (for $n \geq 1$):

$$\sum_{k=0}^{n} (-1)^k \binom{n}{k} = 0 \tag{15}$$

4) Hockey Stick Identity: The sum of consecutive binomial coefficients starting from the boundary equals a single coefficient in the next row:

$$\sum_{i=r}^{n} \binom{i}{r} = \binom{n+1}{r+1} \tag{16}$$

3.3 Algorithmic Construction Paradigms

To construct the first N rows of Pascal's Triangle, we analyze three different algorithmic approaches and contrast their performance.

3.3.1 Approach 1: Naive Recursive Formulation

Based directly on Pascal's Identity, we can implement a recursive method to fetch any coefficient $\binom{n}{k}$:

$$f(n,k) = f(n-1,k-1) + f(n-1,k) \tag{17}$$

Computing an entire row N using this method incurs severe overlapping subproblems. The recursive call tree for $\binom{N}{K}$ matches the structure of the Fibonacci recursion, leading to an exponential time complexity:

$$T_{\text{naive}}(N) = \mathcal{O}(2^N) \tag{18}$$

The space complexity is determined by the maximum depth of the call stack, which is $\mathcal{O}(N)$.

3.3.2 Approach 2: Standard 2D Dynamic Programming

We can avoid recomputing subproblems by storing previously computed coefficients in a 2D grid structure. This is a classic application of dynamic programming.

```
import java.util.ArrayList;
import java.util.List;

public class PascalsTriangle2DDP {
    public static List<List<Integer>> generate(int numRows) {
        List<List<Integer>> triangle = new ArrayList<>();
        if (numRows <= 0) return triangle;

        // Initialize row 0
        List<Integer> firstRow = new ArrayList<>();
        firstRow.add(1);
        triangle.add(firstRow);

        for (int i = 1; i < numRows; i++) {
            List<Integer> row = new ArrayList<>();
            List<Integer> prevRow = triangle.get(i - 1);

            row.add(1); // Boundary element
            for (int j = 1; j < i; j++) {
                // Apply Pascal's Identity
                row.add(prevRow.get(j - 1) + prevRow.get(j));
            }
            row.add(1); // Boundary element
            triangle.add(row);
        }
        return triangle;
    }
}
```

- Time Complexity: We execute a nested loop where the outer loop runs N times and the inner loop runs up to i times. The total number of additions is:

$$\sum_{i=1}^{N-1} (i-1) = \frac{(N-1)(N-2)}{2} = \mathcal{O}(N^2) \tag{19}$$

- Space Complexity: To store the output, we occupy memory proportional to the size of the triangle:

$$S(N) = \sum_{i=1}^{N} i = \frac{N(N+1)}{2} = \mathcal{O}(N^2) \tag{20}$$

3.3.3 Approach 3: Space-Optimized 1D Dynamic Programming

If the goal is to generate only a specific row N of Pascal's Triangle rather than the entire history, we do not need to persist a 2D table. We can update a single 1D array in-place. To prevent overwriting values before they are used in the calculation, we must iterate backwards (from right to left).

```
import java.util.ArrayList;
import java.util.List;

public class PascalsTriangle1DDP {
    public static List<Integer> getRow(int rowIndex) {
        List<Integer> row = new ArrayList<>(rowIndex + 1);
        // Initialize all elements to 0, except the first which is 1
        row.add(1);
        for (int i = 1; i <= rowIndex; i++) {
            row.add(0);
        }

        for (int i = 1; i <= rowIndex; i++) {
            for (int j = i; j >= 1; j--) {
                row.set(j, row.get(j) + row.get(j - 1));
            }
        }
        return row;
    }
}
```

- Time Complexity: Still requires computing the sum of elements, leading to $\mathcal{O}(N^2)$ iterations.
- Space Complexity: Only the 1D state is maintained in memory. Thus:

$$S_{\text{opt}}(N) = \mathcal{O}(N) \quad \text{(excluding the return list space)} \tag{21}$$

4 The Binomial Theorem and Mathematical Proofs

The Binomial Theorem provides a closed-form algebraic expansion for the powers of a binomial expression.

4.1 Theorem Statement

Theorem 6 (The Binomial Theorem). For any real (or complex) variables x and y, and any non-negative integer n:

$$(x+y)^n = \sum_{k=0}^{n} \binom{n}{k} x^{n-k} y^k \tag{22}$$

The general term in the expansion, representing the $(k+1)$-th term, is formulated as:

$$T_{k+1} = \binom{n}{k} x^{n-k} y^k \tag{23}$$

4.2 Inductive Proof

We prove the Binomial Theorem using the Principle of Mathematical Induction on the variable n.

Proof. Base Case ($n = 0$):

$$(x+y)^0 = 1 \quad \text{and} \quad \sum_{k=0}^{0} \binom{0}{k} x^{0-k} y^k = \binom{0}{0} x^0 y^0 = 1 \tag{24}$$

The base case holds.

Inductive Hypothesis: Assume the theorem is true for some positive integer $m \geq 0$:

$$(x+y)^m = \sum_{k=0}^{m} \binom{m}{k} x^{m-k} y^k \tag{25}$$

Inductive Step: We show that the theorem holds for $n = m + 1$:

$$(x+y)^{m+1} = (x+y)(x+y)^m \tag{26}$$

Substituting the induction hypothesis:

$$(x+y)^{m+1} = (x+y)\sum_{k=0}^{m}\binom{m}{k}x^{m-k}y^k \tag{27}$$

$$= x\sum_{k=0}^{m}\binom{m}{k}x^{m-k}y^k + y\sum_{k=0}^{m}\binom{m}{k}x^{m-k}y^k \tag{28}$$

$$= \sum_{k=0}^{m}\binom{m}{k}x^{m-k+1}y^k + \sum_{k=0}^{m}\binom{m}{k}x^{m-k}y^{k+1} \tag{29}$$

For the first sum, we isolate the term for $k = 0$:

$$\sum_{k=0}^{m}\binom{m}{k}x^{m-k+1}y^k = \binom{m}{0}x^{m+1}y^0 + \sum_{k=1}^{m}\binom{m}{k}x^{m-k+1}y^k \tag{30}$$

For the second sum, we make a change of variable by letting $j = k + 1$ (hence $k = j - 1$):

$$\sum_{k=0}^{m}\binom{m}{k}x^{m-k}y^{k+1} = \sum_{j=1}^{m+1}\binom{m}{j-1}x^{m-(j-1)}y^j = \sum_{j=1}^{m}\binom{m}{j-1}x^{m-j+1}y^j + \binom{m}{m}x^0y^{m+1} \tag{31}$$

Since j is a dummy variable of summation, we can replace it with k:

$$= \sum_{k=1}^{m}\binom{m}{k-1}x^{m-k+1}y^k + \binom{m}{m}y^{m+1} \tag{32}$$

Substituting these two decomposed expansions back:

$$(x+y)^{m+1} = \binom{m}{0}x^{m+1} + \sum_{k=1}^{m}\binom{m}{k}x^{m-k+1}y^k + \sum_{k=1}^{m}\binom{m}{k-1}x^{m-k+1}y^k + \binom{m}{m}y^{m+1} \tag{33}$$

$$= \binom{m}{0}x^{m+1} + \sum_{k=1}^{m}\left[\binom{m}{k} + \binom{m}{k-1}\right]x^{m-k+1}y^k + \binom{m}{m}y^{m+1} \tag{34}$$

Applying Pascal's Identity $\binom{m}{k} + \binom{m}{k-1} = \binom{m+1}{k}$:

$$(x+y)^{m+1} = \binom{m}{0}x^{m+1} + \sum_{k=1}^{m}\binom{m+1}{k}x^{m+1-k}y^k + \binom{m}{m}y^{m+1} \tag{35}$$

Note that $\binom{m}{0} = 1 = \binom{m+1}{0}$ and $\binom{m}{m} = 1 = \binom{m+1}{m+1}$. We can integrate the boundary terms into the summation:

$$(x+y)^{m+1} = \binom{m+1}{0}x^{m+1-0}y^0 + \sum_{k=1}^{m}\binom{m+1}{k}x^{m+1-k}y^k + \binom{m+1}{m+1}x^0y^{m+1} \tag{36}$$

$$= \sum_{k=0}^{m+1}\binom{m+1}{k}x^{m+1-k}y^k \tag{37}$$

The formula holds for $n = m + 1$. By the Principle of Mathematical Induction, the Binomial Theorem is true for all integers $n \geq 0$. □

4.3 Combinatorial Proof

Consider the product:

$$(x+y)^n = \underbrace{(x+y)(x+y)\cdots(x+y)}_{n \text{ factors}} \tag{38}$$

When expanding this product, we form terms by choosing either x or y from each of the n factors. A term of the form $x^{n-k}y^k$ arises when we choose y from exactly k factors and x from the remaining $n - k$ factors.

The number of ways to choose k factors from a set of n factors is given by the combination $\binom{n}{k}$. Thus, the coefficient of $x^{n-k}y^k$ in the expanded expression is exactly $\binom{n}{k}$. Summing over all possible values of k from 0 to n yields:

$$(x+y)^n = \sum_{k=0}^{n}\binom{n}{k}x^{n-k}y^k \tag{39}$$

4.4 Middle Terms in Binomial Expansion

Locating the middle term(s) of $(x+y)^n$ depends on the parity of n:

- Case 1: n is even. The expansion contains $n+1$ terms (an odd number). There is a single middle term, which is the $(\frac{n}{2}+1)$-th term:

$$T_{\text{mid}} = T_{\frac{n}{2}+1} = \binom{n}{n/2} x^{n/2} y^{n/2} \tag{40}$$

- Case 2: n is odd. The expansion contains $n+1$ terms (an even number). There are two middle terms, namely the $(\frac{n+1}{2})$-th and $(\frac{n+3}{2})$-th terms:

$$T_{\text{mid},1} = T_{\frac{n+1}{2}} = \binom{n}{\frac{n-1}{2}} x^{\frac{n+1}{2}} y^{\frac{n-1}{2}} \tag{41}$$

$$T_{\text{mid},2} = T_{\frac{n+3}{2}} = \binom{n}{\frac{n+1}{2}} x^{\frac{n-1}{2}} y^{\frac{n+1}{2}} \tag{42}$$

4.5 Constant Terms (Terms Independent of x)

In algebraic expressions of the form $(ax^p + \frac{b}{x^q})^n$, we are often tasked with finding the term that does not contain x (the constant term). We locate this by writing the general term:

$$T_{k+1} = \binom{n}{k} (ax^p)^{n-k} \left(\frac{b}{x^q}\right)^k = \binom{n}{k} a^{n-k} b^k x^{p(n-k)-qk} \tag{43}$$

To make this term independent of x, we set the exponent of x to 0:

$$p(n-k) - qk = 0 \implies pn - (p+q)k = 0 \implies k = \frac{pn}{p+q} \tag{44}$$

If k is a non-negative integer such that $0 \le k \le n$, the constant term exists and is given by T_{k+1}. Otherwise, no constant term exists.

4.6 Rational and Irrational Terms

In expansions of the form $(a^{1/p} + b^{1/q})^n$, where a, b are distinct prime numbers, the terms are rational if and only if the exponents of both terms are integers. The general term is:

$$T_{k+1} = \binom{n}{k} (a^{1/p})^{n-k} (b^{1/q})^k = \binom{n}{k} a^{\frac{n-k}{p}} b^{\frac{k}{q}} \tag{45}$$

For T_{k+1} to be rational, we require:

$$\frac{n-k}{p} \in \mathbb{Z} \quad \text{and} \quad \frac{k}{q} \in \mathbb{Z} \quad \text{for } 0 \le k \le n \tag{46}$$

This means k must be a multiple of q, and $n-k$ must be a multiple of p. Finding the number of rational terms amounts to finding the number of integer values of k that satisfy these simultaneous divisibility constraints. The remaining terms in the expansion $(n+1-N_{\text{rational}})$ are irrational.

4.7 Vandermonde's Identity (Sum of Squares of Coefficients)

Vandermonde's Identity provides a beautiful formulation for the sum of the squares of the binomial coefficients.

Theorem 7 (Vandermonde's Identity for Squares). For any positive integer n:

$$\sum_{k=0}^{n} \binom{n}{k}^2 = \binom{2n}{n} \tag{47}$$

Proof. Consider the identity:

$$(1+x)^{2n} = (1+x)^n (1+x)^n \tag{48}$$

Using the Binomial Theorem, the coefficient of x^n on the left-hand side of the equation is:

$$\text{Coeff of } x^n \text{ in } (1+x)^{2n} = \binom{2n}{n} \tag{49}$$

On the right-hand side, we expand both factors:

$$(1+x)^n (1+x)^n = \left(\sum_{i=0}^{n} \binom{n}{i} x^i\right) \left(\sum_{j=0}^{n} \binom{n}{j} x^j\right) \tag{50}$$

To obtain a term of degree n in the product, we must multiply a term of degree i from the first factor by a term of degree $n-i$ from the second factor. The coefficient of x^n is the sum of these products over all valid i:

$$\text{Coeff of } x^n = \sum_{i=0}^{n} \binom{n}{i}\binom{n}{n-i} \tag{51}$$

By the symmetry property of binomial coefficients, we know that $\binom{n}{n-i} = \binom{n}{i}$. Substituting this in yields:

$$\text{Coeff of } x^n = \sum_{i=0}^{n} \binom{n}{i}\binom{n}{i} = \sum_{i=0}^{n} \binom{n}{i}^2 \tag{52}$$

Equating the coefficients of x^n from both sides:

$$\sum_{k=0}^{n} \binom{n}{k}^2 = \binom{2n}{n} \tag{53}$$

This completes the proof. □

4.8 Numerically Greatest Term (NGT)

For a given real value of x, the terms in the expansion of $(1+x)^n$ vary in magnitude. We want to find the index r that yields the numerically greatest term, T_{r+1}.

We define T_{r+1} to be the numerically greatest term if:

$$|T_{r+1}| \geq |T_r| \quad \text{and} \quad \ldots \tag{54}$$

Using the ratio metric:

$$r \leq m \quad \text{where} \quad m = \frac{(n+1)|x|}{1+|x|} \tag{55}$$

If m is an integer, $|T_m| = |T_{m+1}|$ are both numerically greatest. If m is not an integer, the single greatest term is $T_{\lfloor m \rfloor + 1}$.

4.9 Important Corollaries and Algebraic Identities

By substituting specific values for x and y, we derive standard identities frequently tested in competitive examinations.

Corollary 8 (Sum of Coefficients). Let $x = 1, y = 1$:

$$(1+1)^n = \sum_{k=0}^{n} \binom{n}{k} 1^{n-k} 1^k \implies \sum_{k=0}^{n} \binom{n}{k} = 2^n \tag{56}$$

Corollary 9 (Alternating Sum of Coefficients). Let $x = 1, y = -1$:

$$(1-1)^n = \sum_{k=0}^{n} \binom{n}{k} 1^{n-k}(-1)^k \implies \sum_{k=0}^{n} (-1)^k \binom{n}{k} = 0 \quad \text{for } n \geq 1 \tag{57}$$

Corollary 10 (Derivative-based Identity). Differentiating $(1+x)^n$ and substituting $x = 1$ yields:

$$\sum_{k=1}^{n} k\binom{n}{k} = n \cdot 2^{n-1} \tag{58}$$

Corollary 11 (Integration-based Identity). Integrating $(1+x)^n$ from 0 to 1 yields:

$$\sum_{k=0}^{n} \frac{1}{k+1}\binom{n}{k} = \frac{2^{n+1}-1}{n+1} \tag{59}$$

5 The Multinomial Theorem and Multiset Permutations

The Multinomial Theorem generalizes the Binomial Theorem to expansions of polynomials with m terms.

5.1 Theorem Statement

Theorem 12 (The Multinomial Theorem). For any positive integer n and any real variables $x_1, x_2, \ldots, x_m$:

$$(x_1 + x_2 + \cdots + x_m)^n = \sum_{k_1+k_2+\cdots+k_m=n} \binom{n}{k_1, k_2, \ldots, k_m} \prod_{i=1}^{m} x_i^{k_i} \tag{60}$$

where $k_1 + k_2 + \cdots + k_m = n$ are non-negative integers.

The multinomial coefficient is defined as:

$$\binom{n}{k_1, k_2, \ldots, k_m} = \frac{n!}{k_1!k_2!\cdots k_m!} \tag{61}$$

5.2 Derivation via Multiset Permutations

The multinomial coefficient represents the number of linear permutations of a multiset containing n elements, where there are m distinct types of elements, with k_i elements of the i-th type.

$$\text{Total Permutations} = \binom{n}{k_1}\binom{n-k_1}{k_2}\cdots\binom{n-\sum_{j=1}^{m-1}k_j}{k_m} = \frac{n!}{k_1!k_2!\cdots k_m!} \tag{62}$$

5.3 Sum of Multinomial Coefficients

Setting all $x_i = 1$ in the Multinomial Theorem yields:

$$\sum_{k_1+k_2+\cdots+k_m=n} \binom{n}{k_1, k_2, \ldots, k_m} = m^n \tag{63}$$

5.4 Number of Terms in a Multinomial Expansion

By representing the term count as non-negative integer solutions to $k_1+\cdots+k_m = n$, we use the Stars and Bars combinatorial method to prove:

$$N_{\text{terms}} = \binom{n+m-1}{m-1} \tag{64}$$

6 Modular Combinatorics in Competitive Programming

In competitive programming, calculations are almost always performed modulo a large prime, typically $P = 10^9 + 7$ or $P = 998244353$.

6.1 Fermat's Little Theorem and Modular Division

Using Fermat's Little Theorem, division is converted to multiplication:

$$a^{-1} \equiv a^{P-2} \pmod{P} \tag{65}$$

6.2 Linear Precomputation of Combination Queries

We precompute factorials and inverse factorials up to MAX_N in $\mathcal{O}(N)$ time:

```
public class ModularCombinatorics {
    private static final int MOD = 1_000_000_007;
    private static long[] fact;
    private static long[] invFact;

    public static void precompute(int maxN) {
        fact = new long[maxN + 1];
        invFact = new long[maxN + 1];

        fact[0] = 1;
        for (int i = 1; i <= maxN; i++) {
            fact[i] = (fact[i - 1] * i) % MOD;
        }

        invFact[maxN] = power(fact[maxN], MOD - 2);

        for (int i = maxN - 1; i >= 0; i--) {
            invFact[i] = (invFact[i + 1] * (i + 1)) % MOD;
        }
    }

    public static long nCr(int n, int r) {
        if (r < 0 || r > n) return 0;
        return fact[n] * invFact[r] % MOD * invFact[n - r] % MOD;
    }

    private static long power(long base, long exp) {
        long res = 1;
```

```
        base %= MOD;
        while (exp > 0) {
            if ((exp & 1) == 1) res = (res * base) % MOD;
            base = (base * base) % MOD;
            exp >>= 1;
        }
        return res;
    }
}
```

7 Advanced Counting Sequences: Catalan Numbers, Derangements, and Burnside's Lemma

Beyond direct selection counts, several advanced combinatorial sequences appear constantly in algorithm analysis.

7.1 Catalan Numbers

The n-th Catalan number C_n represents the number of valid configurations of various recursive structures:

$$C_n = \frac{1}{n+1}\binom{2n}{n} = \binom{2n}{n} - \binom{2n}{n-1} \tag{66}$$

Satisfying: $C_0 = 1,\ C_{n+1} = \sum_{i=0}^{n} C_i C_{n-i}$. Used for Dyck Paths, Balanced Parentheses, and unique Binary Search Trees.

7.2 Derangements

A derangement is a permutation of a set of n elements in which no element appears in its original position:

$$D_n = n! \sum_{i=0}^{n} \frac{(-1)^i}{i!}, \quad D_n = (n-1)(D_{n-1} + D_{n-2}) \quad \text{for } n \geq 2 \tag{67}$$

with base cases $D_0 = 1, D_1 = 0$.

7.3 Burnside's Lemma (Symmetry Counting)

For advanced CP and symmetry grouping, we compute the number of distinct orbits under a group of symmetries.

Theorem 13 (Burnside's Lemma). Let G be a finite group of permutations acting on a set X. The number of orbits (distinct symmetric configurations), denoted by $|X/G|$, is:

$$|X/G| = \frac{1}{|G|} \sum_{g \in G} |X^g| \tag{68}$$

where $X^g = \{x \in X \mid g(x) = x\}$ is the set of elements in X kept invariant by g.

For example, the number of unique 2-colored necklace designs of length $N = 4$ under rotation symmetries ($G = \{0°, 90°, 180°, 270°\}$):

$$N_{\text{necklace}} = \frac{1}{4}\left(2^4 + 2^1 + 2^2 + 2^1\right) = \frac{1}{4}\left(16 + 2 + 4 + 2\right) = \frac{24}{4} = 6 \tag{69}$$

8 Lucas' Theorem for Large Combinations

When we need to evaluate $\binom{N}{K} \pmod P$ under huge constraints ($N, K \leq 10^{18}$) but the prime modulo P is relatively small ($P < 10^6$):

$$\binom{N}{K} \equiv \prod_{i=0}^{s} \binom{n_i}{k_i} \pmod P \tag{70}$$

where n_i, k_i are the base-P digits of N and K.

```
public class LucasCombination {
    private static int P;

    private static long smallNCr(int n, int r, long[] fact, long[] invFact) {
        if (r < 0 || r > n) return 0;
        return fact[n] * invFact[r] % P * invFact[n - r] % P;
    }

    public static long lucasNCr(long n, long r, long[] fact, long[] invFact) {
        if (r == 0) return 1;
```

```
        int ni = (int) (n % P);
        int ki = (int) (r % P);
        return (lucasNCr(n / P, r / P, fact, invFact) * smallNCr(ni, ki, fact, invFact)) % P;
    }
}
```

9 Stirling Numbers of the First and Second Kind

In discrete mathematics, partitioning distinct items requires Stirling coefficients.

9.1 Stirling Numbers of the Second Kind

The Stirling number of the second kind, denoted by $S(n, k)$ or $\left\{ {n \atop k} \right\}$, represents the number of ways to partition a set of n labeled elements into k unlabeled, non-empty subsets.

$$S(n, k) = k \cdot S(n-1, k) + S(n-1, k-1) \tag{71}$$

```
public class StirlingNumbers {
    public static long[][] computeStirling(int n, int k) {
        long[][] dp = new long[n + 1][k + 1];
        dp[0][0] = 1;
        for (int i = 1; i <= n; i++) {
            for (int j = 1; j <= Math.min(i, k); j++) {
                dp[i][j] = (j * dp[i - 1][j] + dp[i - 1][j - 1]);
            }
        }
        return dp;
    }
}
```

9.2 Stirling Numbers of the First Kind

The Stirling number of the first kind, denoted by $s(n, k)$ or $\left[{n \atop k} \right]$, represents the number of permutations of n elements containing exactly k disjoint cycles.

$$s(n, k) = (n-1) \cdot s(n-1, k) + s(n-1, k-1) \tag{72}$$

with base cases $s(0, 0) = 1$, $s(n, 0) = s(0, k) = 0$.

10 Integer Partitions

Decomposing an integer into smaller non-negative components is another fundamental dynamic programming and counting structure.

Definition 14 (Integer Partition). The partition function $p(n, k)$ represents the number of ways to write n as a sum of exactly k positive integers, where the order of summands does not matter.

10.1 Recurrence Relation

We compute partitions using the recurrence:

$$p(n, k) = p(n-1, k-1) + p(n-k, k) \tag{73}$$

where: * $p(n-1, k-1)$ represents partitions containing at least one summand equal to 1 (we subtract 1 from the number and 1 from the allowed parts). * $p(n-k, k)$ represents partitions where all parts are strictly greater than 1 (we subtract 1 from each of the k parts, leaving $n-k$ value).

with base cases:

$$p(0, 0) = 1 \tag{74}$$

$$p(n, 0) = 0 \quad \text{for } n > 0 \tag{75}$$

$$p(n, k) = 0 \quad \text{for } k > n \text{ or } k < 0 \tag{76}$$

10.2 Dynamic Programming Implementation

We precompute integer partitions using a 2D dynamic programming grid.

```
public class IntegerPartitions {
    public static long[][] computePartitions(int n, int k) {
        long[][] dp = new long[n + 1][k + 1];
        dp[0][0] = 1;

        for (int i = 1; i <= n; i++) {
            for (int j = 1; j <= k; j++) {
                long term1 = dp[i - 1][j - 1];
                long term2 = (i - j >= 0) ? dp[i - j][j] : 0;
                dp[i][j] = term1 + term2;
            }
        }
        return dp;
    }
}
```

This runs in $\mathcal{O}(N \cdot K)$ time and space.

11 The Inclusion-Exclusion Principle (PIE)

The Principle of Inclusion-Exclusion (PIE) is a general counting technique used to find the cardinality of the union of multiple finite sets.

$$\left|\bigcup_{i=1}^{m} A_i\right| = \sum_{\emptyset \neq J \subseteq \{1,\ldots,m\}} (-1)^{|J|-1} \left|\bigcap_{j \in J} A_j\right| \tag{77}$$

And the complement cardinality: $|\bigcap_{i=1}^{m} A_i^c| = |U| - |\bigcup_{i=1}^{m} A_i|$.

12 The Pigeonhole Principle (PHP)

The Pigeonhole Principle is a simple yet powerful discrete mathematical tool.

- Simple Form: If $n > m$, at least one container must contain more than 1 item.
- Generalized Form: If n items are put into m containers, at least one container must contain at least $\lceil n/m \rceil$ items.

12.1 Subarray Modulo Divisibility Proof

Theorem 15 (Subarray Modulo Divisibility). In any array of N integers, there exists a contiguous subarray whose sum is divisible by N.

Proof. Define the prefix sums $P_i = \sum_{j=0}^{i} A[j] \pmod N$ for $i \in \{0, 1, \ldots, N-1\}$. If any $P_i = 0$, the subarray $A[0 \ldots i]$ is divisible by N. Otherwise, the N prefix sums fall into the remaining $N-1$ pigeonholes $\{1, \ldots, N-1\}$. By the Pigeonhole Principle, at least two prefix sums must be equal: $P_a = P_b$ for $a < b$. Thus:

$$P_b - P_a \equiv 0 \pmod N \implies \sum_{j=a+1}^{b} A[j] \equiv 0 \pmod N \tag{78}$$

Thus, the subarray $A[a+1 \ldots b]$ has a sum divisible by N. □

13 The Twelvefold Way Classification

The Twelvefold Way is a systematic framework used to classify 12 basic problems concerning the counting of distributions of n balls into k boxes.

TABLE 1
The Twelvefold Way: Distributing n balls into k boxes

Balls (size n)	Boxes (size k)	Any Mapping	Injective ($n \leq k$)	Surjective ($n \geq k$)
Distinct	Distinct	k^n	$P(k,n) = \frac{k!}{(k-n)!}$	$k! \cdot S(n,k)$
Identical	Distinct	$\binom{n+k-1}{n}$	$\binom{k}{n}$	$\binom{n-1}{n-k}$
Distinct	Identical	$\sum_{i=1}^{k} S(n,i)$	$[n \leq k]$	$S(n,k)$
Identical	Identical	$p_{\leq k}(n)$	$[n \leq k]$	$p(n,k)$

where: * $S(n,k)$ is the Stirling number of the second kind. * $p(n,k)$ is the integer partition of n into exactly k non-zero parts. * $p_{\leq k}(n) = \sum_{i=1}^{k} p(n,i)$.

14 Exhaustive Problem Walkthroughs

In this section, we walk through detailed solutions for various combinatorics problems found in the GATE Computer Science syllabus and CP.

14.1 Problem 1: Coefficient in Trinomial Expansion

Problem Statement: Find the coefficient of $a^3b^2c^4$ in the expansion of $(2a - 3b + c)^9$.

Solution: Let $x_1 = 2a$, $x_2 = -3b$, and $x_3 = c$ in the trinomial expansion $(x_1 + x_2 + x_3)^9$. The general term in the expansion of $(x_1 + x_2 + x_3)^9$ is:

$$\binom{9}{k_1, k_2, k_3} x_1^{k_1} x_2^{k_2} x_3^{k_3} = \frac{9!}{k_1!k_2!k_3!}(2a)^{k_1}(-3b)^{k_2}(c)^{k_3} \tag{79}$$

subject to $k_1 + k_2 + k_3 = 9$.

We are targeting the term $a^3b^2c^4$, which dictates the exponent values:

$$k_1 = 3, \quad k_2 = 2, \quad k_3 = 4 \tag{80}$$

Checking the constraint: $3 + 2 + 4 = 9$, which is valid. Substituting these values:

$$\begin{aligned}
\text{Target Term} &= \frac{9!}{3!2!4!}(2a)^3(-3b)^2(c)^4 && (81)\\
&= \frac{362880}{(6)(2)(24)} \cdot 8a^3 \cdot 9b^2 \cdot c^4 && (82)\\
&= \frac{362880}{288} \cdot 72a^3b^2c^4 && (83)\\
&= 1260 \cdot 72a^3b^2c^4 && (84)\\
&= 90720a^3b^2c^4 && (85)
\end{aligned}$$

Thus, the coefficient of $a^3b^2c^4$ is 90720.

14.2 Problem 2: Term Count in Polynomial Expansion

Problem Statement: Determine the number of distinct terms in the expansion of $(x + y + z + w)^{10}$.

Solution: The expression is a multinomial of order $n = 10$ with $m = 4$ distinct variables (x, y, z, w). Applying the formula for the number of terms:

$$N_{\text{terms}} = \binom{n + m - 1}{m - 1} = \binom{10 + 4 - 1}{4 - 1} = \binom{13}{3} \tag{86}$$

Evaluating the combination:

$$\binom{13}{3} = \frac{13 \times 12 \times 11}{3 \times 2 \times 1} = \frac{1716}{6} = 286 \tag{87}$$

Thus, there are 286 distinct terms in the expansion.

14.3 Problem 3: Term Independent of x

Problem Statement: Find the term independent of x in the expansion of $\left(3x^2 - \frac{1}{2x}\right)^9$.

Solution: The expression can be framed as a binomial of order $n = 9$ with terms $A = 3x^2$ and $B = -\frac{1}{2x}$. The general term is:

$$\begin{aligned}
T_{k+1} &= \binom{9}{k} A^{9-k} B^k && (88)\\
&= \binom{9}{k} (3x^2)^{9-k} \left(-\frac{1}{2x}\right)^k && (89)\\
&= \binom{9}{k} 3^{9-k} \left(-\frac{1}{2}\right)^k x^{2(9-k)} x^{-k} && (90)\\
&= \binom{9}{k} 3^{9-k} \left(-\frac{1}{2}\right)^k x^{18-3k} && (91)
\end{aligned}$$

For the term to be independent of x, we require the exponent of x to be 0:

$$18 - 3k = 0 \implies 3k = 18 \implies k = 6 \tag{92}$$

Since $k = 6$ is an integer satisfying $0 \leq k \leq 9$, the term independent of x is T_7:

$$T_7 = \binom{9}{6} 3^{9-6} \left(-\frac{1}{2}\right)^6 \tag{93}$$

$$= \binom{9}{3} 3^3 \left(\frac{1}{64}\right) \tag{94}$$

$$= \frac{9 \times 8 \times 7}{6} \cdot 27 \cdot \frac{1}{64} \tag{95}$$

$$= 84 \cdot \frac{27}{64} = \frac{21 \times 27}{16} = \frac{567}{16} \tag{96}$$

Thus, the term independent of x is $\frac{567}{16}$.

14.4 Problem 4: Number of Rational Terms

Problem Statement: Find the number of rational terms in the expansion of $\left(2^{1/3} + 3^{1/5}\right)^{15}$.

Solution: The expansion is of order $n = 15$ with terms $A = 2^{1/3}$ and $B = 3^{1/5}$. The general term is:

$$T_{k+1} = \binom{15}{k} \left(2^{1/3}\right)^{15-k} \left(3^{1/5}\right)^k = \binom{15}{k} 2^{\frac{15-k}{3}} 3^{\frac{k}{5}} \tag{97}$$

For T_{k+1} to be rational, the exponents must be integers:

$$\frac{15-k}{3} \in \mathbb{Z} \quad \text{and} \quad \frac{k}{5} \in \mathbb{Z} \quad \text{for } k \in \{0, 1, \ldots, 15\} \tag{98}$$

From $\frac{k}{5} \in \mathbb{Z}$, the possible values for k are:

$$k \in \{0, 5, 10, 15\} \tag{99}$$

Now, we substitute these values into the first exponent $\frac{15-k}{3}$ to check for integrality:

- If $k = 0$: $\frac{15-0}{3} = 5$ (Integer) $\rightarrow$ Rational
- If $k = 5$: $\frac{15-5}{3} = \frac{10}{3}$ (Not an Integer) $\rightarrow$ Irrational
- If $k = 10$: $\frac{15-10}{3} = \frac{5}{3}$ (Not an Integer) $\rightarrow$ Irrational
- If $k = 15$: $\frac{15-15}{3} = 0$ (Integer) $\rightarrow$ Rational

The only values of k yielding rational terms are $k = 0$ and $k = 15$. Thus, there are exactly 2 rational terms (and $16 - 2 = 14$ irrational terms).

14.5 Problem 5: Numerically Greatest Term

Problem Statement: Find the numerically greatest term in the expansion of $(1 + 3x)^{10}$ when $x = 1/2$.

Solution: Let $y = 3x$. For $x = 1/2$, we have $y = 3(1/2) = 3/2$. We write the expansion as $(1 + y)^{10}$ where $y = 1.5$. Applying the formulation for the numerically greatest term index:

$$m = \frac{(n+1)|y|}{1+|y|} = \frac{(10+1)|1.5|}{1+|1.5|} = \frac{11 \times 1.5}{2.5} = \frac{16.5}{2.5} = 6.6 \tag{100}$$

Since $m = 6.6$ is not an integer, the single numerically greatest term is:

$$T_{\lfloor m \rfloor + 1} = T_{6+1} = T_7 \tag{101}$$

We evaluate the value of T_7:

$$T_7 = \binom{10}{6} 1^{10-6} (y)^6 = \binom{10}{4} (1.5)^6 \tag{102}$$

Evaluating the combination and exponent:

$$\binom{10}{4} = \frac{10 \times 9 \times 8 \times 7}{4 \times 3 \times 2 \times 1} = 210 \tag{103}$$

$$(1.5)^6 = \left(\frac{3}{2}\right)^6 = \frac{729}{64} = 11.390625 \tag{104}$$

Multiplying these values:

$$T_7 = 210 \times \frac{729}{64} = \frac{105 \times 729}{32} = \frac{76545}{32} \approx 2392.03125 \tag{105}$$

Thus, the numerically greatest term is T_7, and its value is $\frac{76545}{32}$.

14.6 Problem 6: Modular Combinatorics Calculation

Problem Statement: Find $\binom{1000}{300} \pmod{10^9+7}$ using modular factorial arithmetic, given the precomputed factorial database.

Solution: The combination is $\frac{1000!}{300! \cdot 700!} \pmod{10^9+7}$. By precomputing factorials and inverse factorials modulo 10^9+7:

$$\binom{1000}{300} \equiv \text{fact}[1000] \cdot \text{invFact}[300] \cdot \text{invFact}[700] \pmod{10^9+7} \tag{106}$$

These values are queried in $\mathcal{O}(1)$ time from the array compiled in Section VI-B, resolving division using Fermat's multiplicative inverse.

14.7 Problem 7: Binary Trees and Catalan Numbers

Problem Statement: Find the number of distinct Binary Search Trees (BSTs) that can be constructed with 5 unique keys.

Solution: The number of distinct BSTs with n unique keys corresponds to the n-th Catalan number C_n. For $n = 5$:

$$C_5 = \frac{1}{6}\binom{10}{5} \tag{107}$$

Evaluating the combination:

$$\binom{10}{5} = \frac{10 \times 9 \times 8 \times 7 \times 6}{5 \times 4 \times 3 \times 2 \times 1} = 252 \tag{108}$$

Calculating C_5:

$$C_5 = \frac{252}{6} = 42 \tag{109}$$

Thus, exactly 42 unique BSTs can be constructed.

14.8 Problem 8: Derangement Permutations

Problem Statement: There are 4 letters addressed to 4 different people. Find the number of ways to insert the letters into the envelopes such that no letter goes into the correct envelope.

Solution: This is a direct application of Derangements (D_n) for $n = 4$:

$$D_4 = 4!\left(1 - \frac{1}{1!} + \frac{1}{2!} - \frac{1}{3!} + \frac{1}{4!}\right) \tag{110}$$

Evaluating the fractions:

$$D_4 = 24\left(1 - 1 + \frac{1}{2} - \frac{1}{6} + \frac{1}{24}\right) \tag{111}$$

$$= 24\left(\frac{12 - 4 + 1}{24}\right) \tag{112}$$

$$= 24\left(\frac{9}{24}\right) = 9 \tag{113}$$

Thus, there are exactly 9 complete derangements.

14.9 Problem 9: Lucas' Theorem Evaluation

Problem Statement: Evaluate $\binom{10}{3} \pmod 3$ using Lucas' Theorem.

Solution: We are using prime $P = 3$. We write $N = 10$ and $K = 3$ in base 3:

$$10 = 1 \cdot 3^2 + 0 \cdot 3^1 + 1 \cdot 3^0 \implies (101)_3 \tag{114}$$

$$3 = 0 \cdot 3^2 + 1 \cdot 3^1 + 0 \cdot 3^0 \implies (010)_3 \tag{115}$$

Applying Lucas' Theorem:

$$\binom{10}{3} \equiv \binom{1}{0} \cdot \binom{0}{1} \cdot \binom{1}{0} \pmod 3 \tag{116}$$

$$\equiv 1 \cdot 0 \cdot 1 \pmod 3 \tag{117}$$

$$\equiv 0 \pmod 3 \tag{118}$$

Let us check this by evaluating $\binom{10}{3}$ directly:

$$\binom{10}{3} = \frac{10 \times 9 \times 8}{6} = 120 \tag{119}$$

Dividing 120 by 3:

$$120 = 3 \times 40 + 0 \implies 120 \equiv 0 \pmod 3 \tag{120}$$

The result is correct and verified.

14.10 Problem 10: Stirling Partitioning

Problem Statement: Find the number of ways to partition 5 distinct students into 3 identical groups such that no group is empty.

Solution: This is a direct application of the Stirling Number of the Second Kind, $S(n,k)$ for $n=5$ and $k=3$. Applying the recurrence relation:

$$S(5,3) = 3 \cdot S(4,3) + S(4,2) \tag{121}$$

We expand the subproblems:

$$S(4,3) = 3 \cdot S(3,3) + S(3,2) = 3(1) + S(3,2) \tag{122}$$

$$S(3,2) = 2 \cdot S(2,2) + S(2,1) = 2(1) + 1 = 3 \implies S(4,3) = 3 + 3 = 6 \tag{123}$$

$$S(4,2) = 2 \cdot S(3,2) + S(3,1) = 2(3) + 1 = 7 \tag{124}$$

Now, substitute these back:

$$S(5,3) = 3(6) + 7 = 18 + 7 = 25 \tag{125}$$

Thus, there are exactly 25 ways to partition the students.

14.11 Problem 11: Inclusion-Exclusion Counting

Problem Statement: Find the number of integers from 1 to 1000 (inclusive) that are not divisible by 2, 3, or 5.

Solution: Let $U = \{1, 2, \ldots, 1000\}$, so $|U| = 1000$. Let A_1, A_2, A_3 be the sets of integers in U divisible by 2, 3, and 5, respectively. We calculate their individual sizes:

$$|A_1| = \left\lfloor \frac{1000}{2} \right\rfloor = 500, \quad |A_2| = \left\lfloor \frac{1000}{3} \right\rfloor = 333, \quad |A_3| = \left\lfloor \frac{1000}{5} \right\rfloor = 200 \tag{126}$$

Now, we calculate the sizes of double intersections:

$$|A_1 \cap A_2| = \left\lfloor \frac{1000}{6} \right\rfloor = 166 \tag{127}$$

$$|A_1 \cap A_3| = \left\lfloor \frac{1000}{10} \right\rfloor = 100 \tag{128}$$

$$|A_2 \cap A_3| = \left\lfloor \frac{1000}{15} \right\rfloor = 66 \tag{129}$$

Now, we calculate the triple intersection:

$$|A_1 \cap A_2 \cap A_3| = \left\lfloor \frac{1000}{30} \right\rfloor = 33 \tag{130}$$

Applying the Principle of Inclusion-Exclusion, the union size is:

$$|A_1 \cup A_2 \cup A_3| = (500 + 333 + 200) - (166 + 100 + 66) + 33 \tag{131}$$

$$= 1033 - 332 + 33 = 734 \tag{132}$$

The number of integers not divisible by 2, 3, or 5 is the complement size:

$$N = 1000 - 734 = 266 \tag{133}$$

Thus, there are 266 such integers.

14.12 Problem 12: Pigeonhole Principle Divisibility

Problem Statement: Prove that in any set of 5 integers, there exist two whose difference is divisible by 4.

Solution: Let $S = \{x_1, x_2, x_3, x_4, x_5\}$ be a set of 5 integers. We map each integer to its remainder modulo 4. The possible remainders are $\{0, 1, 2, 3\}$, which represents $m = 4$ pigeonholes. Since we have $n = 5$ items and $m = 4$ containers, and $5 > 4$, by the Pigeonhole Principle, at least two integers in S must share the same remainder modulo 4. Let these integers be x_a and x_b for $a \neq b$. Thus:

$$x_a \equiv x_b \pmod 4 \implies x_a - x_b \equiv 0 \pmod 4 \tag{134}$$

This means the difference $x_a - x_b$ is divisible by 4. The proof is complete.

15 Conclusion

This paper has presented a rigorous formal analysis of the fundamental principles of combinatorics: counting rules, selection coefficients, Pascal's Triangle, and the Binomial and Multinomial Theorems. We investigated three computational approaches for Pascal's Triangle construction, proving that a space-optimized dynamic programming algorithm reduces the operational memory footprint from $\mathcal{O}(N^2)$ to $\mathcal{O}(N)$.

By analyzing multinomial expansions, multiset permutations, Catalan sequences, derangements, rotational symmetries under Burnside's Lemma, Stirling cycle and set partitions, integer decompositions, the Inclusion-Exclusion Principle, and Pigeonhole divisions under the Twelvefold Way distribution matrix, we established mathematical structures to calculate algebraic coefficients and predict terms in polynomial systems. These tools form the baseline for analyzing algorithms and resolving complex counting challenges in computer science.